

class 08 mini project

Omar Siddiqui (PID: A19093654)

Overview

This mini-project applies unsupervised learning (PCA + hierarchical clustering) to real breast cancer biopsy data to see if we can distinguish malignant from benign samples without using the diagnosis labels.

- Data = measurements of cell nuclei from fine needle aspiration (FNA) biopsies
- 569 samples, 30 features per sample
- Goal: can unsupervised methods recover the malignant/benign grouping on their own?

Key idea: We're intentionally ignoring the diagnosis labels during analysis — that's what makes this *unsupervised*. We use the labels only at the end to check how well we did.

2. Exploratory Data Analysis

Preparing the Data

Notes: - First column `diagnosis` is the “answer key” — we remove it before analysis so we don't cheat - Store diagnosis separately as a factor so we can use it for plotting/checking later - `row.names=1` sets the patient ID as the row name instead of a column

```
fna.data <- "WisconsinCancer.csv"
wisc.df <- read.csv(fna.data, row.names=1)

head(wisc.df, 4)
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
842302	M	17.99	10.38	122.80	1001.0
842517	M	20.57	17.77	132.90	1326.0
84300903	M	19.69	21.25	130.00	1203.0
84348301	M	11.42	20.38	77.58	386.1
	smoothness_mean	compactness_mean	concavity_mean	concave.points_mean	
842302	0.11840	0.27760	0.3001		0.14710
842517	0.08474	0.07864	0.0869		0.07017
84300903	0.10960	0.15990	0.1974		0.12790
84348301	0.14250	0.28390	0.2414		0.10520
	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	perimeter_se
842302	0.2419	0.07871	1.0950	0.9053	8.589
842517	0.1812	0.05667	0.5435	0.7339	3.398
84300903	0.2069	0.05999	0.7456	0.7869	4.585
84348301	0.2597	0.09744	0.4956	1.1560	3.445
	area_se	smoothness_se	compactness_se	concavity_se	concave.points_se
842302	153.40	0.006399	0.04904	0.05373	0.01587
842517	74.08	0.005225	0.01308	0.01860	0.01340
84300903	94.03	0.006150	0.04006	0.03832	0.02058
84348301	27.23	0.009110	0.07458	0.05661	0.01867
	symmetry_se	fractal_dimension_se	radius_worst	texture_worst	
842302	0.03003	0.006193	25.38	17.33	
842517	0.01389	0.003532	24.99	23.41	
84300903	0.02250	0.004571	23.57	25.53	
84348301	0.05963	0.009208	14.91	26.50	
	perimeter_worst	area_worst	smoothness_worst	compactness_worst	
842302	184.60	2019.0	0.1622	0.6656	
842517	158.80	1956.0	0.1238	0.1866	
84300903	152.50	1709.0	0.1444	0.4245	
84348301	98.87	567.7	0.2098	0.8663	
	concavity_worst	concave.points_worst	symmetry_worst		
842302	0.7119	0.2654	0.4601		
842517	0.2416	0.1860	0.2750		
84300903	0.4504	0.2430	0.3613		
84348301	0.6869	0.2575	0.6638		
	fractal_dimension_worst				
842302	0.11890				
842517	0.08902				
84300903	0.08758				
84348301	0.17300				

```
# Remove diagnosis column - we don't use this in unsupervised analysis
wisc.data <- wisc.df[,-1]

# Save diagnosis as a factor for later checking
diagnosis <- as.factor(wisc.df$diagnosis)
```

Q1: How many observations are in this dataset?

```
nrow(wisc.data)
```

```
[1] 569
```

569 observations (patients).

Q2: How many of the observations have a malignant diagnosis?

```
table(diagnosis)
```

```
diagnosis
  B    M
357 212
```

```
sum(diagnosis == "M")
```

```
[1] 212
```

212 malignant diagnoses.

Q3: How many variables/features in the data are suffixed with `_mean`?

```
sum(grepl("_mean", colnames(wisc.data)))
```

```
[1] 10
```

10 features end with `_mean`.

3. Principal Component Analysis

Performing PCA

Notes: - Need to scale here because features use different units and have very different variances
- Without scaling, features with large absolute values (like area) would dominate the PCA unfairly - Always check `colMeans()` and `apply(..., sd)` first to confirm scaling is needed

```
colMeans(wisc.data)
```

radius_mean	texture_mean	perimeter_mean
1.412729e+01	1.928965e+01	9.196903e+01
area_mean	smoothness_mean	compactness_mean
6.548891e+02	9.636028e-02	1.043410e-01
concavity_mean	concave.points_mean	symmetry_mean
8.879932e-02	4.891915e-02	1.811619e-01
fractal_dimension_mean	radius_se	texture_se
6.279761e-02	4.051721e-01	1.216853e+00
perimeter_se	area_se	smoothness_se
2.866059e+00	4.033708e+01	7.040979e-03
compactness_se	concavity_se	concave.points_se
2.547814e-02	3.189372e-02	1.179614e-02
symmetry_se	fractal_dimension_se	radius_worst
2.054230e-02	3.794904e-03	1.626919e+01
texture_worst	perimeter_worst	area_worst
2.567722e+01	1.072612e+02	8.805831e+02
smoothness_worst	compactness_worst	concavity_worst
1.323686e-01	2.542650e-01	2.721885e-01
concave.points_worst	symmetry_worst	fractal_dimension_worst
1.146062e-01	2.900756e-01	8.394582e-02

```
apply(wisc.data, 2, sd)
```

radius_mean	texture_mean	perimeter_mean
3.524049e+00	4.301036e+00	2.429898e+01
area_mean	smoothness_mean	compactness_mean
3.519141e+02	1.406413e-02	5.281276e-02
concavity_mean	concave.points_mean	symmetry_mean
7.971981e-02	3.880284e-02	2.741428e-02
fractal_dimension_mean	radius_se	texture_se
7.060363e-03	2.773127e-01	5.516484e-01
perimeter_se	area_se	smoothness_se
2.021855e+00	4.549101e+01	3.002518e-03
compactness_se	concavity_se	concave.points_se
1.790818e-02	3.018606e-02	6.170285e-03
symmetry_se	fractal_dimension_se	radius_worst
8.266372e-03	2.646071e-03	4.833242e+00
texture_worst	perimeter_worst	area_worst
6.146258e+00	3.360254e+01	5.693570e+02
smoothness_worst	compactness_worst	concavity_worst
2.283243e-02	1.573365e-01	2.086243e-01
concave.points_worst	symmetry_worst	fractal_dimension_worst
6.573234e-02	6.186747e-02	1.806127e-02

```
# Scale = TRUE because features have very different units and variances
wisc.pr <- prcomp(wisc.data, scale=TRUE)
summary(wisc.pr)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	3.6444	2.3857	1.67867	1.40735	1.28403	1.09880	0.82172
Proportion of Variance	0.4427	0.1897	0.09393	0.06602	0.05496	0.04025	0.02251
Cumulative Proportion	0.4427	0.6324	0.72636	0.79239	0.84734	0.88759	0.91010
	PC8	PC9	PC10	PC11	PC12	PC13	PC14
Standard deviation	0.69037	0.6457	0.59219	0.5421	0.51104	0.49128	0.39624
Proportion of Variance	0.01589	0.0139	0.01169	0.0098	0.00871	0.00805	0.00523
Cumulative Proportion	0.92598	0.9399	0.95157	0.9614	0.97007	0.97812	0.98335
	PC15	PC16	PC17	PC18	PC19	PC20	PC21
Standard deviation	0.30681	0.28260	0.24372	0.22939	0.22244	0.17652	0.1731
Proportion of Variance	0.00314	0.00266	0.00198	0.00175	0.00165	0.00104	0.0010
Cumulative Proportion	0.98649	0.98915	0.99113	0.99288	0.99453	0.99557	0.9966
	PC22	PC23	PC24	PC25	PC26	PC27	PC28

Standard deviation	0.16565	0.15602	0.1344	0.12442	0.09043	0.08307	0.03987
Proportion of Variance	0.00091	0.00081	0.0006	0.00052	0.00027	0.00023	0.00005
Cumulative Proportion	0.99749	0.99830	0.9989	0.99942	0.99969	0.99992	0.99997
	PC29	PC30					
Standard deviation	0.02736	0.01153					
Proportion of Variance	0.00002	0.00000					
Cumulative Proportion	1.00000	1.00000					

Q4: What proportion of the original variance is captured by PC1?

PC1 captures 44.27% of the original variance.

Q5: How many PCs are required to describe at least 70% of the original variance?

3 PCs (cumulative proportion reaches 72.6% at PC3).

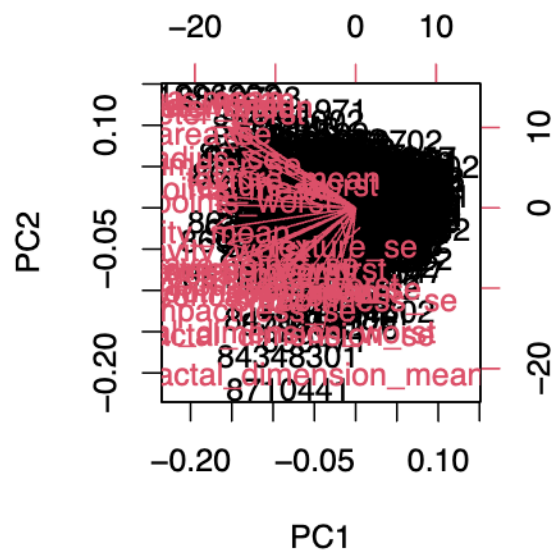
Q6: How many PCs are required to describe at least 90% of the original variance?

7 PCs (cumulative proportion reaches 91.0% at PC7).

Interpreting PCA Results

Notes: - Biplots look cool but are basically unreadable with real datasets that have many observations - A simple PC1 vs PC2 scatter plot colored by diagnosis is much more informative - If the two diagnoses separate cleanly on PC1, that tells us PC1 is capturing the cancer signal

```
biplot(wisc.pr)
```

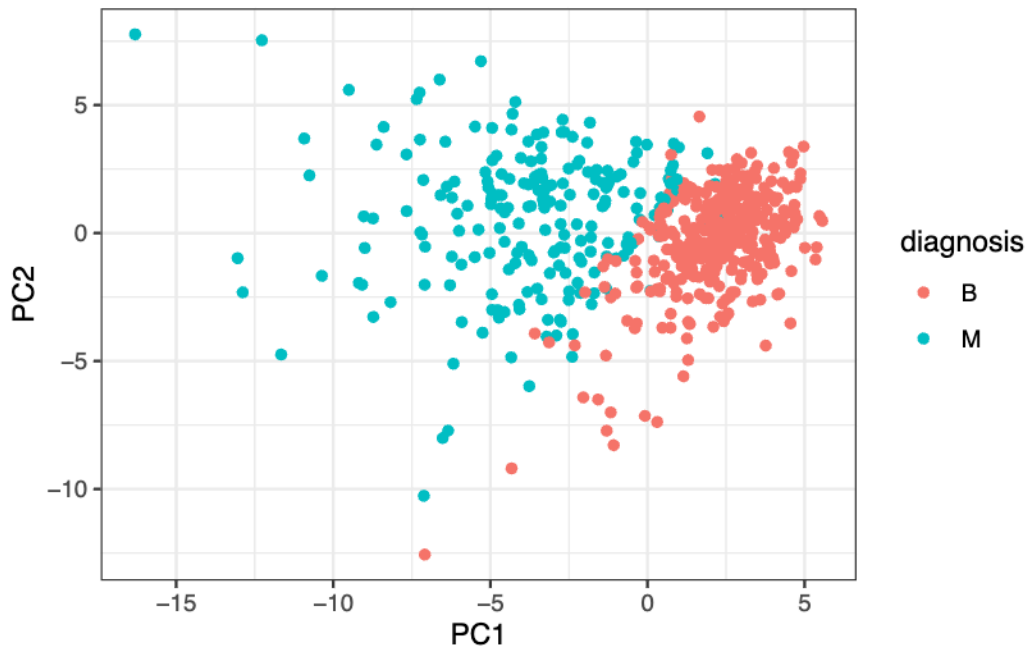


Q7: What stands out to you about this plot? Is it easy or difficult to understand? Why?

It's extremely difficult to understand. With 569 observations all plotted as row names, the plot is completely unreadable — everything overlaps. This is why we switch to a simpler scatter plot below.

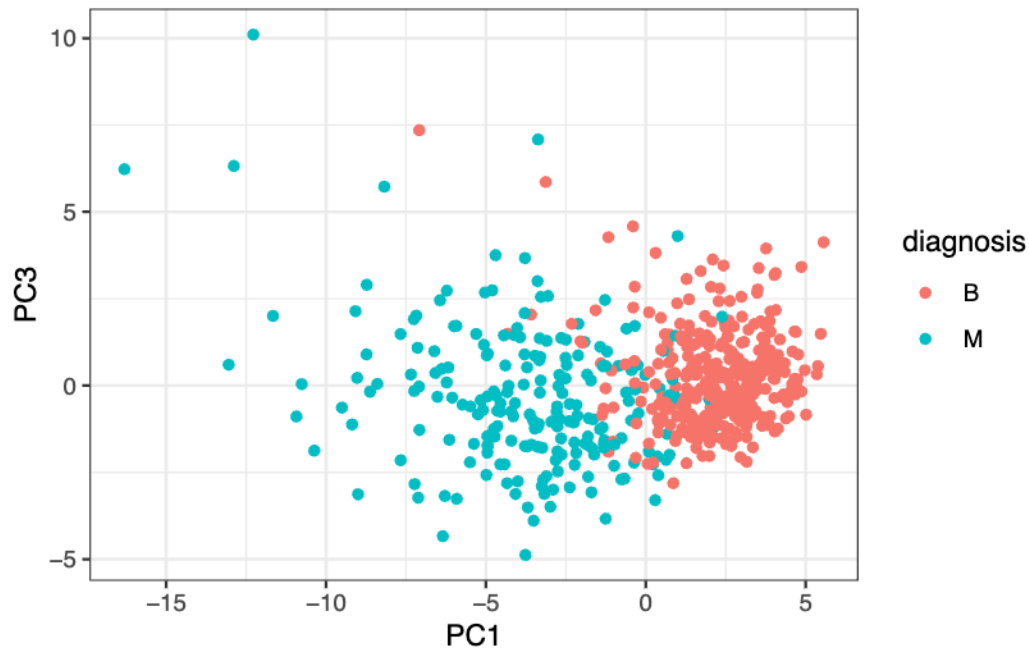
```
library(ggplot2)

# PC1 vs PC2 colored by diagnosis
ggplot(as.data.frame(wisc.pr$x)) +
  aes(x = PC1, y = PC2, col = diagnosis) +
  geom_point() +
  theme_bw()
```



Q8: Generate a similar plot for PC1 vs PC3. What do you notice about these plots?

```
ggplot(as.data.frame(wisc.pr$x)) +
  aes(x = PC1, y = PC3, col = diagnosis) +
  geom_point() +
  theme_bw()
```



PC1 vs PC2 shows a cleaner separation between malignant and benign than PC1 vs PC3. PC2 captures more diagnostically relevant variance than PC3 does. In both plots PC1 is doing most of the heavy lifting for separating the two groups.

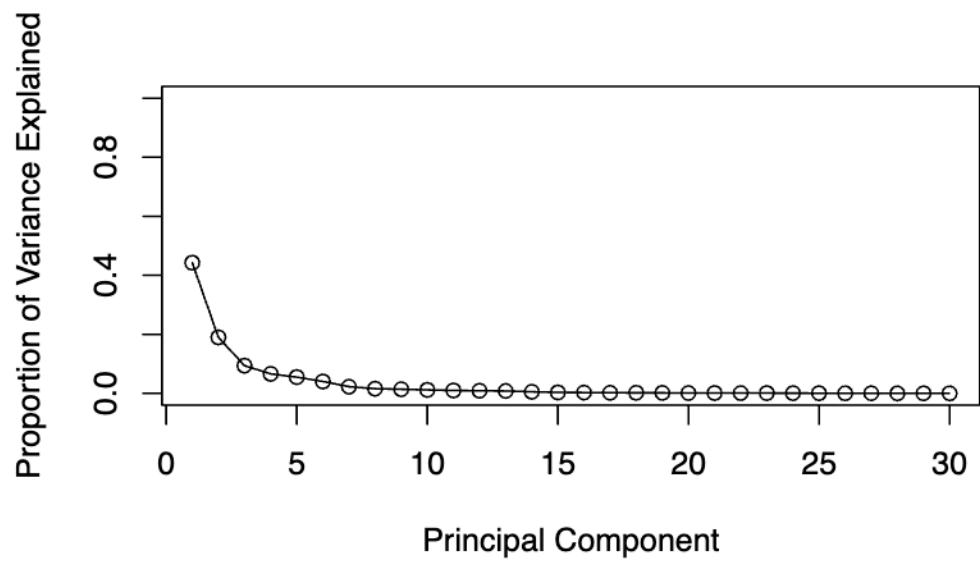
Variance Explained

```
# Variance of each PC
pr.var <- wisc.pr$sdev^2
head(pr.var)
```

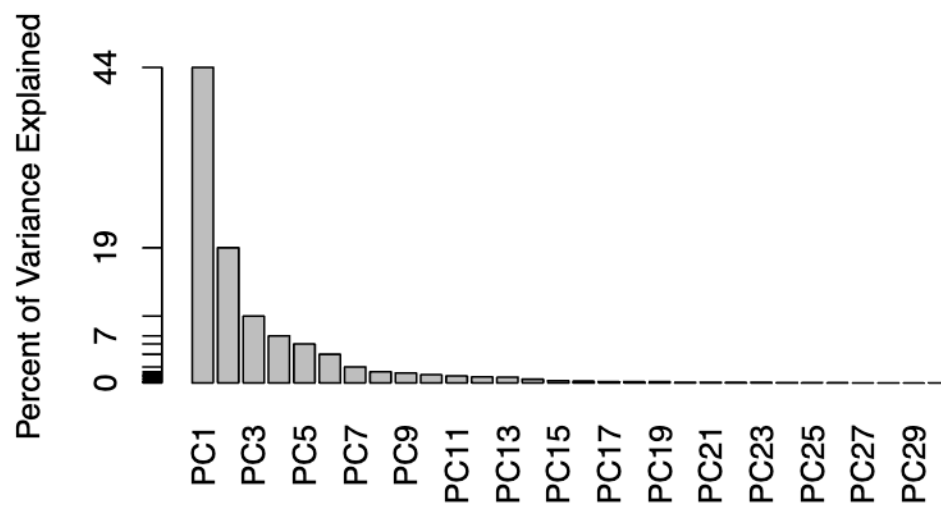
```
[1] 13.281608  5.691355  2.817949  1.980640  1.648731  1.207357
```

```
# Proportion of variance explained
pve <- pr.var / sum(pr.var)

# Line plot
plot(pve, xlab="Principal Component",
     ylab="Proportion of Variance Explained",
     ylim=c(0,1), type="o")
```



```
# Barplot
barplot(pve, ylab="Percent of Variance Explained",
        names.arg=paste0("PC", 1:length(pve)), las=2, axes=FALSE)
axis(2, at=pve, labels=round(pve,2)*100)
```

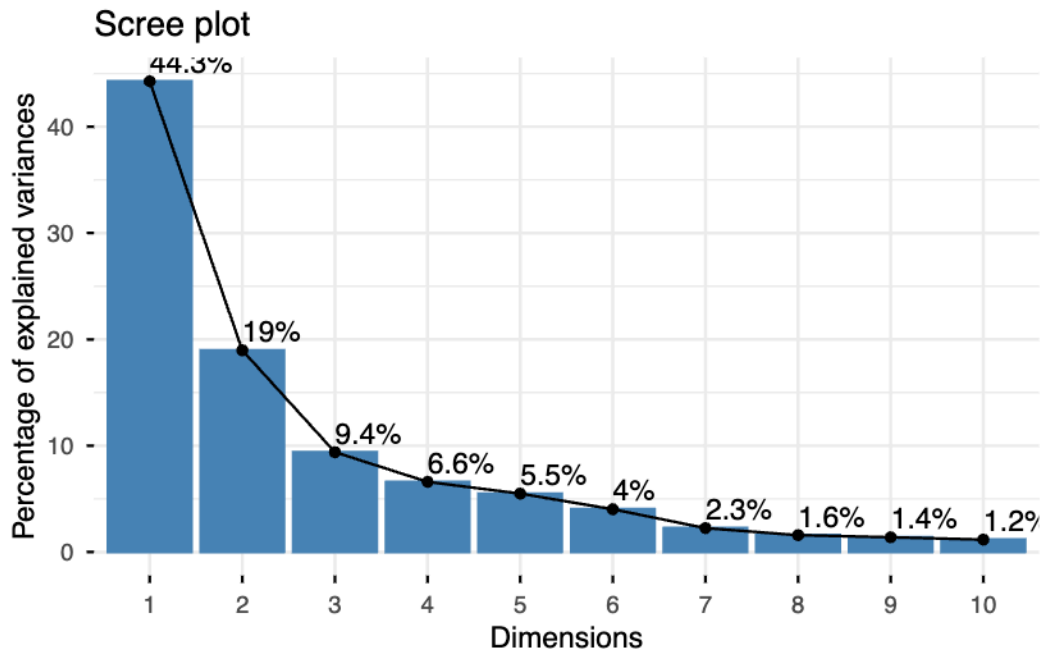


```
# Optional ggplot scree plot with factoextra  
# install.packages("factoextra")  
library(factoextra)
```

Welcome to factoextra!

Want to learn more? See two factoextra-related books at <https://www.datanovia.com/en/product-guide-to-principal-component-methods-in-r/>

```
fviz_eig(wisc.pr, addlabels=TRUE)
```



Q9: For PC1, what is the loading score for concave.points_mean? Are any features larger?

```
wisc.pr$rotation["concave.points_mean", 1]
```

```
[1] -0.2608538
```

```
# Check all loadings for PC1 sorted by magnitude
sort(abs(wisc.pr$rotation[,1]), decreasing=TRUE)
```

concave.points_mean	concavity_mean	concave.points_worst
0.26085376	0.25840048	0.25088597
compactness_mean	perimeter_worst	concavity_worst
0.23928535	0.23663968	0.22876753
radius_worst	perimeter_mean	area_worst
0.22799663	0.22753729	0.22487053

area_mean	radius_mean	perimeter_se
0.22099499	0.21890244	0.21132592
compactness_worst	radius_se	area_se
0.21009588	0.20597878	0.20286964
concave.points_se	compactness_se	concavity_se
0.18341740	0.17039345	0.15358979
smoothness_mean	symmetry_mean	fractal_dimension_worst
0.14258969	0.13816696	0.13178394
smoothness_worst	symmetry_worst	texture_worst
0.12795256	0.12290456	0.10446933
texture_mean	fractal_dimension_se	fractal_dimension_mean
0.10372458	0.10256832	0.06436335
symmetry_se	texture_se	smoothness_se
0.04249842	0.01742803	0.01453145

The loading for *concave.points_mean* on PC1 is approximately -0.261. Several other features have larger magnitude contributions to PC1, including *concave.points_worst*, *perimeter_worst*, and *radius_worst*.

4. Hierarchical Clustering

Notes: - Hierarchical clustering doesn't require specifying k in advance — it builds a full tree - We then cut the tree at whatever height gives us the number of clusters we want - **complete** linkage = distance between clusters defined by their two farthest points - Scaling is important here for the same reason as PCA

```
# Scale the data
data.scaled <- scale(wisc.data)

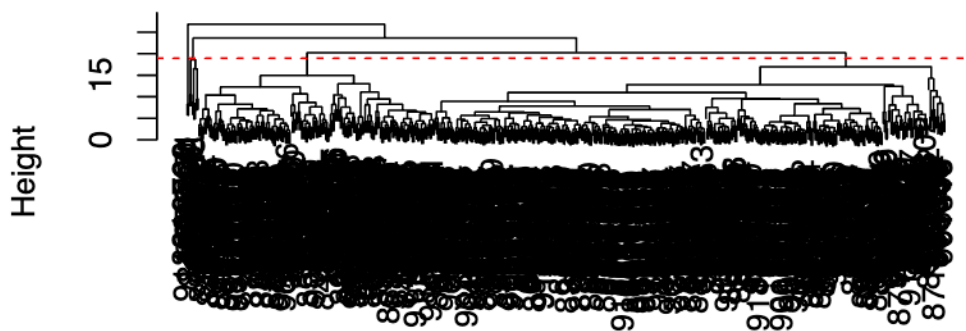
# Calculate distance matrix
data.dist <- dist(data.scaled)

# Hierarchical clustering with complete linkage
wisc.hclust <- hclust(data.dist, method="complete")
```

Q10: Using `plot()` and `abline()`, what is the height at which the clustering model has 4 clusters?

```
plot(wisc.hclust)
abline(h=19, col="red", lty=2)
```

Cluster Dendrogram



```
data.dist
hclust (*, "complete")
```

At a height of approximately 19, the dendrogram splits into 4 clusters.

Selecting Number of Clusters

```
wisc.hclust.clusters <- cutree(wisc.hclust, k=4)
table(wisc.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.hclust.clusters	B	M
1	12	165
2	2	5
3	343	40
4	0	2

Q11: OPTIONAL — Can you find a better cluster vs diagnosis match by cutting into 2–6 clusters?

```
# Try different k values
for(k in 2:6) {
  cat("\nk =", k, "\n")
  print(table(cutree(wisc.hclust, k=k), diagnosis))
}
```

```
k = 2
  diagnosis
    B    M
1 357 210
2   0    2
```

```
k = 3
  diagnosis
    B    M
1 355 205
2   2    5
3   0    2
```

```
k = 4
  diagnosis
    B    M
1  12 165
2   2    5
3 343  40
4   0    2
```

```
k = 5
  diagnosis
    B    M
1  12 165
2   0    5
3 343  40
```

4	2	0
5	0	2


```

k = 6
  diagnosis
    B   M
1  12 165
2   0   5
3 331  39
4   2   0
5  12   1
6   0   2

```

k=2 gives the simplest split but mixes many benign into the malignant cluster. k=4 gives a reasonable separation with cluster 1 mostly malignant and cluster 3 mostly benign, though not perfect. There's no clean separation regardless of k when clustering on raw data.

Q12: Which linkage method gives your favorite results? Explain your reasoning.

```

wisc.hclust.complete <- hclust(data.dist, method="complete")
wisc.hclust.average  <- hclust(data.dist, method="average")
wisc.hclust.single   <- hclust(data.dist, method="single")
wisc.hclust.ward      <- hclust(data.dist, method="ward.D2")

# Compare 4-cluster cuts
table(cutree(wisc.hclust.complete, k=4), diagnosis)

```

	diagnosis	
	B	M
1	12	165
2	2	5
3	343	40
4	0	2

```
table(cutree(wisc.hclust.ward, k=4), diagnosis)
```

	diagnosis	
	B	M
1	0	115
2	6	48
3	337	48
4	14	1

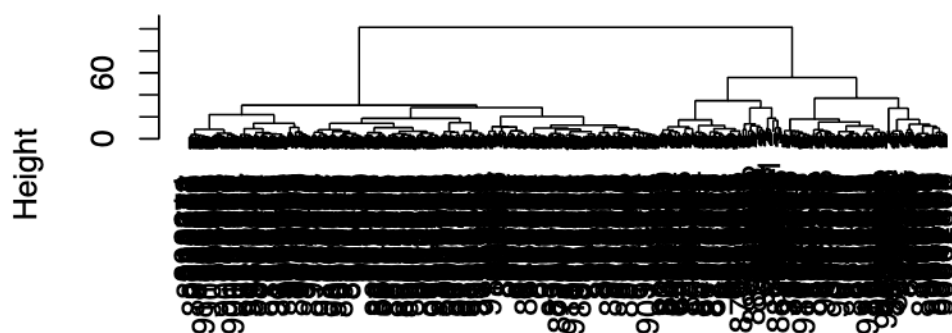
ward.D2 gives the best results. It minimizes within-cluster variance at each merge step, producing more compact and balanced clusters. With $k=2$, ward.D2 gives a much cleaner separation of malignant vs benign compared to the other methods.

5. Combining Methods — Clustering on PCA Results

Notes: - Instead of clustering on all 30 raw features, we cluster on the first 7 PCs (which capture 90%+ of variance) - This removes noise from lower PCs and gives clustering a cleaner signal to work with - ward.D2 is especially appropriate here since PCA and Ward's both work with variance

```
# Use first 7 PCs (capturing >90% variance) with ward.D2
wisc.pr.hclust <- hclust(dist(wisc.pr$x[, 1:7]), method="ward.D2")
plot(wisc.pr.hclust)
```

Cluster Dendrogram



```
dist(wisc.pr$x[, 1:7])
hclust (*, "ward.D2")
```

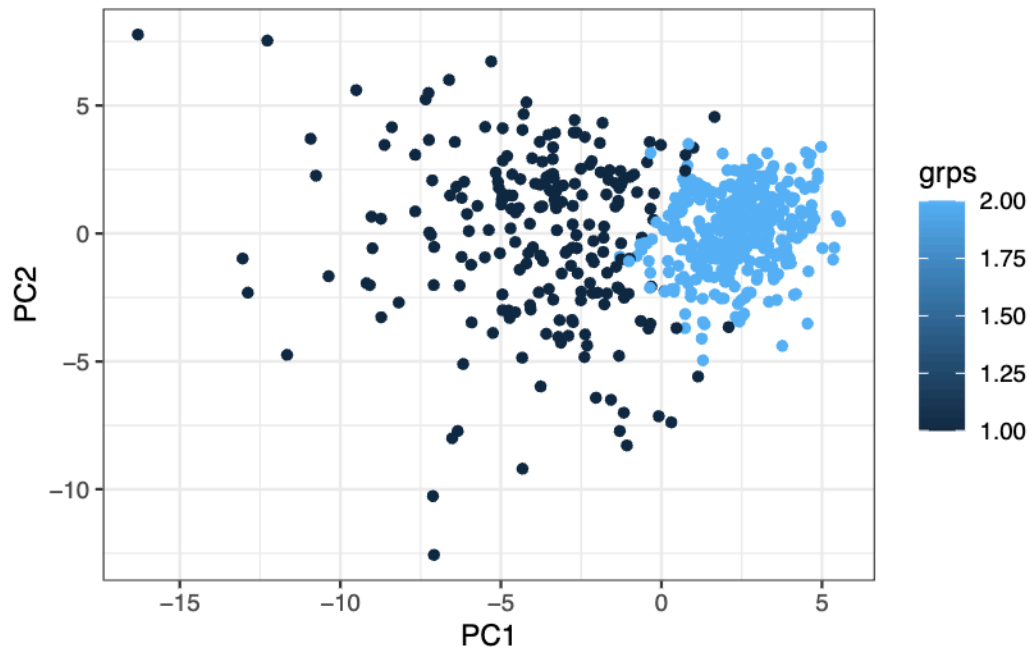
```
# Cut into 2 clusters
grps <- cutree(wisc.pr.hclust, k=2)
table(grps)
```

```
grps
  1   2
216 353
```

```
table(grps, diagnosis)
```

```
      diagnosis
grps    B    M
  1   28 188
  2  329   24
```

```
# Plot PC1 vs PC2 colored by cluster
ggplot(as.data.frame(wisc.pr$x)) +
  aes(PC1, PC2, col=grps) +
  geom_point() +
  theme_bw()
```



```
wisc.pr.hclust.clusters <- cutree(wisc.pr.hclust, k=2)
```

Q13: How well does the PCA + hclust model separate the two diagnoses?

```
table(wisc.pr.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.pr.hclust.clusters	B	M
1	28	188
2	329	24

Much better than clustering on raw data alone. Cluster 1 captures 188 of 212 malignant samples (89% sensitivity) and cluster 2 captures 329 of 357 benign samples (92% specificity). The PCA preprocessing clearly improved the clustering performance.

Q14: How do the raw hierarchical clustering models compare to the PCA + hclust model?

```
table(wisc.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.hclust.clusters	B	M
1	12	165
2	2	5
3	343	40
4	0	2

```
table(wisc.pr.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.pr.hclust.clusters	B	M
1	28	188
2	329	24

The raw hierarchical clustering (4 clusters) does a poorer job — it splits into 4 uneven groups and has more mixed clusters. The PCA + hclust model with just 2 clusters gives a cleaner and more interpretable separation of malignant vs benign, showing that PCA preprocessing meaningfully improved the result.

6. Sensitivity / Specificity

Notes: - **Sensitivity** = $TP / (TP + FN)$ — how well does the model catch actual malignant cases? - **Specificity** = $TN / (TN + FP)$ — how well does it correctly identify benign cases? - Missing a malignant case (false negative) is worse in a clinical setting than a false positive

Q15: OPTIONAL — Which procedure had the best specificity? Best sensitivity?

```
# PCA + hclust (cluster 1 = malignant)
tp <- 188; fn <- 24; tn <- 329; fp <- 28
sensitivity_pca <- tp / (tp + fn)
specificity_pca <- tn / (tn + fp)
cat("PCA+hclust Sensitivity:", round(sensitivity_pca, 3), "\n")
```

PCA+hclust Sensitivity: 0.887

```
cat("PCA+hclust Specificity:", round(specificity_pca, 3), "\n")
```

PCA+hclust Specificity: 0.922

```
# Raw hclust (cluster 1 = malignant)
tp2 <- 165; fn2 <- 47; tn2 <- 343; fp2 <- 14
sensitivity_raw <- tp2 / (tp2 + fn2)
specificity_raw <- tn2 / (tn2 + fp2)
cat("Raw hclust Sensitivity:", round(sensitivity_raw, 3), "\n")
```

Raw hclust Sensitivity: 0.778

```
cat("Raw hclust Specificity:", round(specificity_raw, 3), "\n")
```

Raw hclust Specificity: 0.961

PCA + hclust has better sensitivity (0.887 vs 0.778) — it catches more of the malignant cases. Raw hclust has slightly better specificity (0.961 vs 0.922) but misses far more malignant samples, which is the worse error to make clinically.

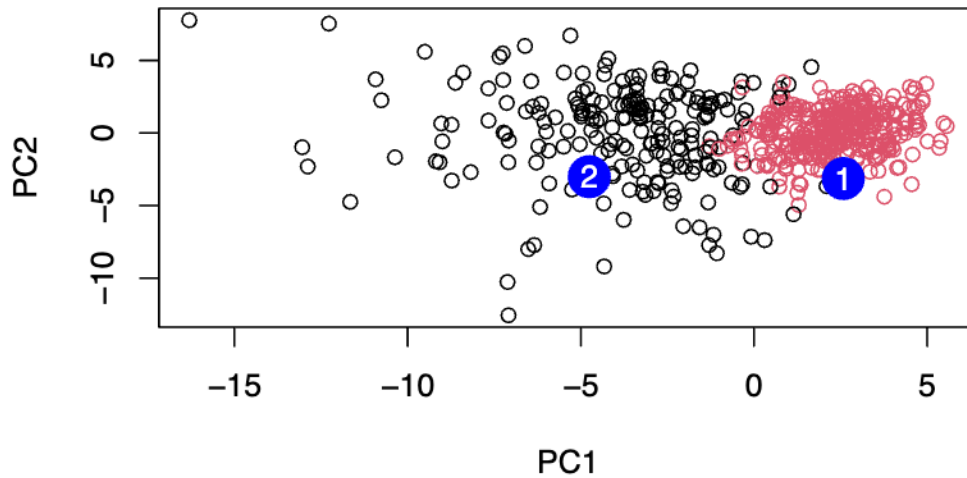
7. Prediction

Notes: - `predict()` on a `prcomp` object projects new data into the existing PCA space - This is powerful — you train the model once and can classify new patients without rerunning PCA
- We then just look at where the new patients land relative to the known malignant/benign clusters

```
url <- "https://tinyurl.com/new-samples-CSV"
new <- read.csv(url)
npc <- predict(wisc.pr, newdata=new)
npc
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
[1,]	2.576616	-3.135913	1.3990492	-0.7631950	2.781648	-0.8150185	-0.3959098
[2,]	-4.754928	-3.009033	-0.1660946	-0.6052952	-1.140698	-1.2189945	0.8193031
	PC8	PC9	PC10	PC11	PC12	PC13	PC14
[1,]	-0.2307350	0.1029569	-0.9272861	0.3411457	0.375921	0.1610764	1.187882
[2,]	-0.3307423	0.5281896	-0.4855301	0.7173233	-1.185917	0.5893856	0.303029
	PC15	PC16	PC17	PC18	PC19	PC20	
[1,]	0.3216974	-0.1743616	-0.07875393	-0.11207028	-0.08802955	-0.2495216	
[2,]	0.1299153	0.1448061	-0.40509706	0.06565549	0.25591230	-0.4289500	
	PC21	PC22	PC23	PC24	PC25	PC26	
[1,]	0.1228233	0.09358453	0.08347651	0.1223396	0.02124121	0.078884581	
[2,]	-0.1224776	0.01732146	0.06316631	-0.2338618	-0.20755948	-0.009833238	
	PC27	PC28	PC29	PC30			
[1,]	0.220199544	-0.02946023	-0.015620933	0.005269029			
[2,]	-0.001134152	0.09638361	0.002795349	-0.019015820			

```
plot(wisc.pr$x[,1:2], col=grps)
points(npc[,1], npc[,2], col="blue", pch=16, cex=3)
text(npc[,1], npc[,2], c(1,2), col="white")
```



Q16: Which of the new patients should we prioritize for follow up?

Patient 2 should be prioritized. They project into the cluster 1 region of the PCA plot, which corresponds to the predominantly malignant group. Patient 1 falls in the benign cluster region and is lower priority for immediate follow up.

8. Summary

In this mini-project we applied PCA + hierarchical clustering to real breast cancer biopsy data:

- PCA reduced 30 features down to 7 PCs capturing >90% of variance
- Clustering on PCA scores outperformed clustering on raw data
- The combined approach correctly separated most malignant from benign samples without ever using the diagnosis labels

- New patient samples can be projected onto the PCA space for rapid classification

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.4.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

other attached packages:
[1] factoextra_2.0.0 ggplot2_4.0.2

loaded via a namespace (and not attached):
 [1] gtable_0.3.6      jsonlite_2.0.0    dplyr_1.2.1       compiler_4.5.3
 [5] ggsignif_0.6.4    tidysselect_1.2.1 Rcpp_1.1.1         tidyr_1.3.2
 [9] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0     R6_2.6.1
[13] ggpubr_0.6.3      labeling_0.4.3    generics_0.1.4    Formula_1.2-5
[17] knitr_1.51        backports_1.5.1   ggrepel_0.9.8     tibble_3.3.1
[21] car_3.1-5         pillar_1.11.1     RColorBrewer_1.1-3 rlang_1.1.7
[25] broom_1.0.12      xfun_0.57         S7_0.2.1          otel_0.2.0
[29] cli_3.6.5         withr_3.0.2       magrittr_2.0.5    digest_0.6.39
[33] grid_4.5.3        rstudioapi_0.18.0 lifecycle_1.0.5   vctrs_0.7.2
[37] rstatix_0.7.3     evaluate_1.0.5    glue_1.8.0        farver_2.1.2
[41] abind_1.4-8       carData_3.0-6     rmarkdown_2.31    purrr_1.2.2
[45] tools_4.5.3       pkgconfig_2.0.3   htmltools_0.5.9
```